

---

# Data Cleaning Script Template

Student Name: Nicole Riley

Dataset: CERT Error Rate Data from FY 2021-2025

---

```
In [1]: # Import libraries -  
# pandas will be able to do most of my CSV cleaning tasks  
  
import pandas as pd
```

I want to be able to compare multiple years of data, so I will load in data sets for fiscal years 2021-2025

```
In [2]: # Loading data files -  
  
df_2021 = pd.read_csv('Medicare_FFS_CERT_2021.csv')  
df_2022 = pd.read_csv('Medicare_FFS_CERT_2022.csv')  
df_2023 = pd.read_csv('Medicare_FFS_CERT_2023.csv')  
df_2024 = pd.read_csv('Medicare_FFS_CERT_2024.csv')  
df_2025 = pd.read_csv('2025_CERT_Public_Data.csv')
```

## Explore the Data

I'll first take a look at the a few of the primary features of the 2025 data set, including the number of rows and columns, the column names, and how many columns contain null values.

```
In [3]: print(df_2025.shape)  
print(df_2025.info())  
df_2025.sample(20)
```

```
(163940, 8)
<class 'pandas.DataFrame'>
RangeIndex: 163940 entries, 0 to 163939
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   claim_control_number                 163940 non-null  int64
1   Part                                163940 non-null  str
2   DRG                                  8750 non-null    float64
3   HCPCS Procedure Code                 117082 non-null  str
4   Provider Type                       163940 non-null  str
5   Type of Bill                        119682 non-null  str
6   Review Decision                     163940 non-null  str
7   Error Code                          163940 non-null  str
dtypes: float64(1), int64(1), str(6)
memory usage: 10.0 MB
None
```

Out[3]:

|               | claim_control_number | Part                                                    | DRG   | HCPSC<br>Procedure<br>Code | Provider Type                                     | Type<br>of<br>Bill | Re<br>Deci |
|---------------|----------------------|---------------------------------------------------------|-------|----------------------------|---------------------------------------------------|--------------------|------------|
| <b>70804</b>  | 2538068              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN   | G0151                      | HHA                                               | 329                | Disa       |
| <b>20680</b>  | 2537039              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN   | NaN                        | Inpatient Critical<br>Access Hospital             | 111                | A          |
| <b>134114</b> | 2559454              | 1. Part B                                               | NaN   | 77063                      | Diagnostic<br>Radiology                           | NaN                | A          |
| <b>155452</b> | 2567543              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN   | G0152                      | HHA                                               | 329                | A          |
| <b>95944</b>  | 2556088              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN   | 82310                      | ESRD                                              | 721                | A          |
| <b>40121</b>  | 2543545              | 1. Part B                                               | NaN   | 63685                      | Physical Medicine<br>and Rehabilitation           | NaN                | A          |
| <b>113305</b> | 2557543              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS)              | 840.0 | NaN                        | DRG Short Term                                    | 11G                | A          |
| <b>110728</b> | 2556873              | 1. Part B                                               | NaN   | 98980                      | Registered<br>Dietician/Nutrition<br>Professional | NaN                | A          |
| <b>51837</b>  | 2543606              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN   | 90999                      | ESRD                                              | 721                | A          |
| <b>6379</b>   | 2534467              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN   | NaN                        | ESRD                                              | 721                | A          |
| <b>8016</b>   | 2536818              | 1. Part B                                               | NaN   | 88381                      | Clinical Laboratory<br>(Billing<br>Independently) | NaN                | A          |

|               | claim_control_number | Part                                                    | DRG | HCPCS<br>Procedure<br>Code | Provider Type                                              | Type<br>of<br>Bill | Re<br>Deci |
|---------------|----------------------|---------------------------------------------------------|-----|----------------------------|------------------------------------------------------------|--------------------|------------|
| <b>57489</b>  | 2546739              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN | G0156                      | Hospice                                                    | 813                | A          |
| <b>48685</b>  | 2543341              | 1. Part B                                               | NaN | 81307                      | Clinical Laboratory<br>(Billing<br>Independently)          | NaN                | Disa       |
| <b>151902</b> | 2568239              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN | G0156                      | Hospice                                                    | 813                | A          |
| <b>139688</b> | 2565478              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN | NaN                        | ESRD                                                       | 721                | A          |
| <b>96439</b>  | 2556749              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN | J1720                      | OPPS, Laboratory<br>(an FI),<br>Ambulatory<br>(Billing ... | 131                | A          |
| <b>156923</b> | 2566244              | 1. Part B                                               | NaN | 83036                      | Clinical Laboratory<br>(Billing<br>Independently)          | NaN                | A          |
| <b>38396</b>  | 2540256              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN | NaN                        | Inpatient<br>Rehabilitation<br>Hospitals                   | 111                | A          |
| <b>36846</b>  | 2542261              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN | 96374                      | OPPS, Laboratory<br>(an FI),<br>Ambulatory<br>(Billing ... | 131                | A          |
| <b>2825</b>   | 2534456              | 3. Part<br>A(Excluding<br>Inpatient<br>Hospital<br>PPS) | NaN | NaN                        | ESRD                                                       | 721                | A          |

I see some interesting things emerging. It appears that the DRG column contains quite a few null values, with only 8,750 of the dataframe's almost 164,000 rows containing data. I also see that the Type of Bill column is missing about 50,000 values, as is the HCPCS procedure code column. I'll want to look into this further.

First, though, I know I want to be able to compare this data across the five years I've loaded. I believe the best way to do this is to combine them into a single dataframe with a new column indicating the fiscal year. Before I can do that, I need to make sure that the data is consistent across all five files.

```
In [4]: # I want make sure that all of my dataframes have the same columns
# I'll use a for loop to print the columns from each dataframe to a list so I can s
# if they all look the same, I should be able to combine them without needing to do

for df in [df_2021, df_2022, df_2023, df_2024, df_2025]:
    print(df.columns.tolist())
```

```
['claim_control_number', 'Part', 'DRG', 'HCPCS Procedure Code', 'Provider Type', 'Ty
pe of Bill', 'Review Decision', 'Error Code']
['claim_control_number', 'Part', 'DRG', 'HCPCS Procedure Code', 'Provider Type', 'Ty
pe of Bill', 'Review Decision', 'Error Code']
['claim_control_number', 'Part', 'DRG', 'HCPCS Procedure Code', 'Provider Type', 'Ty
pe of Bill', 'Review Decision', 'Error Code']
['claim_control_number', 'Part', 'DRG', 'HCPCS Procedure Code', 'Provider Type', 'Ty
pe of Bill', 'Review Decision', 'Error Code']
['claim_control_number', 'Part', 'DRG', 'HCPCS Procedure Code', 'Provider Type', 'Ty
pe of Bill', 'Review Decision', 'Error Code']
```

It looks consistent, so I shouldn't have any problems with the columns lining up appropriately.

However, it does look like the claim control number is the only column header currently in snake case, with all other column headers in title case with spaces by default. I will change the claim control number column header to title case with spaces as well, for consistency and Power BI readability.

I also want to make sure all these columns contain the same type of data, and that the type of data makes sense for the contents, as well as for the kind of analysis I may want to perform.

```
In [5]: # I'll use another for loop to print the data types in all of my dataframes
# I'll use the zip function to add a list of the corresponding years so I can refer
for year, df in zip([2021, 2022, 2023, 2024, 2025], [df_2021, df_2022, df_2023, df_
    print(f"\n{year}:")
    print(df.dtypes)
```

2021:  
claim\_control\_number       int64  
Part                        str  
DRG                         float64  
HCPCS Procedure Code       str  
Provider Type               str  
Type of Bill                str  
Review Decision             str  
Error Code                  str  
dtype: object

2022:  
claim\_control\_number       int64  
Part                        str  
DRG                         float64  
HCPCS Procedure Code       str  
Provider Type               str  
Type of Bill                str  
Review Decision             str  
Error Code                  str  
dtype: object

2023:  
claim\_control\_number       int64  
Part                        str  
DRG                         float64  
HCPCS Procedure Code       str  
Provider Type               str  
Type of Bill                str  
Review Decision             str  
Error Code                  str  
dtype: object

2024:  
claim\_control\_number       int64  
Part                        str  
DRG                         float64  
HCPCS Procedure Code       str  
Provider Type               str  
Type of Bill                str  
Review Decision             str  
Error Code                  str  
dtype: object

2025:  
claim\_control\_number       int64  
Part                        str  
DRG                         float64  
HCPCS Procedure Code       str  
Provider Type               str  
Type of Bill                str  
Review Decision             str  
Error Code                  str  
dtype: object

It looks like the types are consistent across all five data sets, so I should not run into issues when I go to combine them.

One thing I notice is that the DRG column contains float values. I know that Medicare DRG numbers should be three-digit numbers. I'll investigate further by filtering one dataframe to only show data where the value in the DRG column is not null, then take a random sample.

```
In [6]: df_2025[df_2025['DRG'].notnull()].sample(25)
```

Out[6]:

|               | claim_control_number | Part                                       | DRG   | HCPSC<br>Procedure<br>Code | Provider<br>Type     | Type<br>of<br>Bill | Review<br>Decision |    |
|---------------|----------------------|--------------------------------------------|-------|----------------------------|----------------------|--------------------|--------------------|----|
| <b>146946</b> | 2564454              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 853.0 | NaN                        | DRG<br>Short<br>Term | 111                | Agree              |    |
| <b>137588</b> | 2560801              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 85.0  | NaN                        | DRG<br>Short<br>Term | 111                | Agree              |    |
| <b>63613</b>  | 2537454              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 629.0 | NaN                        | DRG<br>Short<br>Term | 111                | Disagree           |    |
| <b>103163</b> | 2551389              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 274.0 | NaN                        | DRG<br>Short<br>Term | 111                | Disagree           | Do |
| <b>78456</b>  | 2549569              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 454.0 | NaN                        | DRG<br>Short<br>Term | 111                | Agree              |    |
| <b>126304</b> | 2557447              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 57.0  | NaN                        | DRG<br>Short<br>Term | 110                | Agree              |    |
| <b>66005</b>  | 2547913              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 673.0 | NaN                        | DRG<br>Short<br>Term | 111                | Agree              |    |
| <b>38052</b>  | 2540037              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 259.0 | NaN                        | DRG<br>Short<br>Term | 111                | Disagree           |    |
| <b>25437</b>  | 2535017              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 84.0  | NaN                        | DRG<br>Short<br>Term | 110                | Agree              |    |
| <b>163108</b> | 2568890              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 545.0 | NaN                        | DRG<br>Short<br>Term | 111                | Agree              |    |
| <b>162033</b> | 2568477              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 617.0 | NaN                        | DRG<br>Short<br>Term | 111                | Agree              |    |



|  | claim_control_number | Part    | DRG                                        | HCPSC<br>Procedure<br>Code | Provider<br>Type | Type<br>of<br>Bill   | Review<br>Decision |             |
|--|----------------------|---------|--------------------------------------------|----------------------------|------------------|----------------------|--------------------|-------------|
|  | 106503               | 2554893 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 562.0                      | NaN              | DRG<br>Short<br>Term | 111                | Disagree    |
|  | 20925                | 2537179 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 274.0                      | NaN              | DRG<br>Short<br>Term | 111                | Disagree Do |
|  | 158116               | 2566529 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 884.0                      | NaN              | DRG<br>Short<br>Term | 111                | Disagree    |
|  | 137648               | 2560920 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 270.0                      | NaN              | DRG<br>Short<br>Term | 117                | Agree       |
|  | 126247               | 2557892 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 392.0                      | NaN              | DRG<br>Short<br>Term | 111                | Disagree    |
|  | 147118               | 2564738 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 872.0                      | NaN              | DRG<br>Short<br>Term | 111                | Agree       |
|  | 35832                | 2539933 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 331.0                      | NaN              | DRG<br>Short<br>Term | 111                | Agree       |
|  | 145768               | 2564174 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 183.0                      | NaN              | DRG<br>Short<br>Term | 111                | Disagree    |
|  | 134860               | 2559017 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 806.0                      | NaN              | DRG<br>Short<br>Term | 111                | Agree       |
|  | 38609                | 2539711 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 641.0                      | NaN              | DRG<br>Short<br>Term | 111                | Agree       |
|  | 103083               | 2551847 | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 637.0                      | NaN              | DRG<br>Short<br>Term | 111                | Agree       |

|               | claim_control_number | Part                                       | DRG   | HCPSC<br>Procedure<br>Code | Provider<br>Type             | Type<br>of<br>Bill | Review<br>Decision |
|---------------|----------------------|--------------------------------------------|-------|----------------------------|------------------------------|--------------------|--------------------|
| <b>134508</b> | 2559463              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 470.0 | NaN                        | DRG<br>Short<br>Term         | 111                | Agree              |
| <b>26294</b>  | 2538548              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 493.0 | NaN                        | DRG<br>Short<br>Term         | 111                | Agree              |
| <b>85611</b>  | 2549021              | 4. Part<br>A(Inpatient<br>Hospital<br>PPS) | 884.0 | NaN                        | Other FI<br>Service<br>Types | 111                | Agree              |

The DRG column presents an interesting challenge. It contains null values for the vast majority of records, as it is only relevant for inpatient hospital PPS claims. However, for that inpatient data the DRG information could offer valuable insight into error trends by diagnosis.

Before dropping the column from the primary dataset, I will first create a separate inpatient subset containing only Part A/IPPS records where DRG data is present. This subset will retain the DRG column and serve as the basis for a focused inpatient analysis.

Once the subset has been created, the DRG column will be dropped from the primary combined dataset to keep it clean and avoid complications from a predominantly null column.

Upon reviewing the preview, I'm noticing another inconsistency I will want to address: the Part column contains a number before the description of the Medicare part. I want to see what these values look like.

```
In [7]: # preview the contents of the Part column with .unique()

df_2025['Part'].unique()
```

```
Out[7]: <StringArray>
[
      '2. DME MAC',
      '1. Part B',
      '3. Part A(Excluding Inpatient Hospital PPS)',
      '4. Part A(Inpatient Hospital PPS)']
Length: 4, dtype: str
```

I want to be able to use the Part values for sorting in Power BI, and the prefix numbers at the beginning of the string are not providing any additional meaning, so I will strip them out of the final file.

## Extra Context - DRG Reference File

Since I know I want to do additional analysis on the inpatient hospital / DRG data, I want to pull information about what the DRG numbers actually mean. The DRG reference terminology was sourced from the Tuva Project, an open-source healthcare data platform widely used by healthcare data professionals.

*Note: MS-DRG descriptions may vary slightly across fiscal years. The Tuva reference file reflects current MS-DRG assignments and is applied uniformly across all years in this dataset.*

*See README for full source documentation and citation.*

```
In [8]: # I need to load in the DRG reference set from Tuva and preview it
DRG_ref = pd.read_csv('ms_drg.csv_0_0_0.csv')

DRG_ref.sample(15)
```

Out[8]:

|            | 001 | \N | P  | Heart Transplant or Implant of Heart Assist System with MCC | 0 | \N.1       |
|------------|-----|----|----|-------------------------------------------------------------|---|------------|
| <b>786</b> | 131 | \N | \N | Cranial/facial procedures w CC/MCC                          | 1 | 2020-10-01 |
| <b>761</b> | 976 | 25 | M  | HIV with Major Related Condition without CC/MCC             | 0 | \N         |
| <b>50</b>  | 63  | 01 | M  | Ischemic Stroke, Precerebral Occlusion or Tran...           | 0 | \N         |
| <b>204</b> | 265 | 05 | P  | AICD Lead Procedures                                        | 0 | \N         |
| <b>649</b> | 817 | 14 | P  | Other Antepartum Diagnoses with O.R. Procedure...           | 0 | \N         |
| <b>386</b> | 475 | 08 | P  | Amputation for Musculoskeletal System and Conn...           | 0 | \N         |
| <b>38</b>  | 42  | 01 | P  | Peripheral, Cranial Nerve and Other Nervous Sy...           | 0 | \N         |
| <b>451</b> | 554 | 08 | M  | Bone Diseases and Arthropathies without MCC                 | 0 | \N         |
| <b>258</b> | 321 | 05 | P  | Percutaneous Cardiovascular Procedures with In...           | 0 | \N         |
| <b>211</b> | 272 | 05 | P  | Other Major Cardiovascular Procedures without ...           | 0 | \N         |
| <b>811</b> | 491 | \N | \N | Back & neck proc exc spinal fusion w/o CC/MCC               | 1 | 2014-10-01 |
| <b>505</b> | 623 | 10 | P  | Skin Grafts and Wound Debridement for Endocrin...           | 0 | \N         |
| <b>553</b> | 688 | 11 | M  | Kidney and Urinary Tract Neoplasms without CC/MCC           | 0 | \N         |
| <b>644</b> | 812 | 16 | M  | Red Blood Cell Disorders without MCC                        | 0 | \N         |
| <b>613</b> | 776 | 14 | M  | Postpartum and Post Abortion Diagnoses without...           | 0 | \N         |

Upon loading in the Tuva DRG reference file, I encountered two issues that required additional handling.

First, the dataset does not contain column headings. The Tuva Project provides column documentation for this file on their website. Using this documentation, I was able to assign the following column names by repeating the loading process: `ms_drg_code`, `mdc_code`, `medical_surgical`, `ms_drg_description`, `deprecated`, and `deprecated_date`. I will convert these column headers to title case with spaces in order to be consistent with the primary data set.

Second, the file contains `\N` values in several columns. This is a MySQL-style null indicator commonly used when exporting CSV files from a MySQL database. Rather than leaving these as-is, I will replace them with `NaN` to maintain consistency with the null representation used throughout the rest of my dataset. I'll need `numpy` for this, so I will import this library now.

Finally, the last two columns contain information about DRG code depreciation. This is more complexity than is required for this exploratory dashboard, so these two columns will be dropped.

```
In [9]: import numpy as np
```

## Based on the exploration above, the following cleaning and preparation steps will be performed:

- Add a fiscal\_year column to each CERT dataset and concatenate into a single combined file
- Normalize the column headers in the combined file to title case with spaces
- Remove the prefix number from the Part column to improve readability and sortability
- Create a separate inpatient subset containing only Part A/IPPS claims where DRG data is present
- Cast the DRG column to string in the inpatient subset
- Clean the Tuva DRG reference file by adding normalized title case with spaced headers, replacing nulls, and dropping deprecation information columns
- Merge the cleaned Tuva DRG reference file with the inpatient subset for additional clinical context
- Drop the DRG column from the combined dataset
- Export both cleaned datasets as CSV files

---

## Data Cleaning Script

---

```
In [10]: # add a Fiscal Year column for identification of the data year after union

for name, df in zip([2021, 2022, 2023, 2024, 2025], [df_2021, df_2022, df_2023, df_2024, df_2025]):
    df['Fiscal Year'] = name

In [11]: # combine the files into a single CERT dataframe

full_CERT_df = pd.concat([df_2021, df_2022, df_2023, df_2024, df_2025])

In [12]: # normalize column names to title case with spaces

full_CERT_df = full_CERT_df.rename(columns={'claim_control_number': 'Claim Control Number'})

In [13]: # remove numeric prefixes from Part column

full_CERT_df['Part'] = full_CERT_df['Part'].str.split('.', n=1).str[1]

In [14]: # create DRG subset dataframe

DRG_subset_df = full_CERT_df[full_CERT_df['DRG'].notnull()]

In [15]: # in DRG subset, convert DRG column to integer for reference data set merge
```

```
DRG_subset_df['DRG'] = DRG_subset_df['DRG'].astype(int)
```

```
In [16]: # reload Tuva reference data with column headers, normalized to title case with spa
DRG_ref = pd.read_csv('ms_drg.csv_0_0_0.csv', header=None, names=['MS-DRG Code', 'M
```

```
In [17]: # drop the deprecated and deprecated date columns from reference df
DRG_ref = DRG_ref.drop(columns=['deprecated', 'deprecated_date'])
```

```
In [18]: # replace special character with NaN
DRG_ref = DRG_ref.replace(r'\N', np.nan)
```

```
In [19]: # merge cleaned Tuva data into the DRG subset
DRG_subset_df = pd.merge(DRG_subset_df, DRG_ref, left_on='DRG', right_on='MS-DRG Code'
```

```
In [20]: # drop extra DRG column, and convert the remaining DRG column to string type to en
DRG_subset_df = DRG_subset_df.drop(columns=['DRG'])
DRG_subset_df['MS-DRG Code'] = DRG_subset_df['MS-DRG Code'].astype(str)
```

```
In [21]: # drop DRG column from primary dataframe
full_CERT_df = full_CERT_df.drop(columns='DRG')
```

---

## Save the Cleaned Datasets

---

```
In [22]: full_CERT_df.to_csv("cleaned_data_full_CERT.csv", index=False)
DRG_subset_df.to_csv("cleaned_data_DRG_sub.csv", index=False)
```

---

## Additional Cleaning Steps

After working with the data and creating visualizations in Power BI, additional review and cleaning steps were necessary. These steps have been detailed below.

---

```
In [23]: # while building error rate measures, discovered there was one row with null in the
full_CERT_df[full_CERT_df['Review Decision'].isnull()]
```

Out[23]:

|        | Claim<br>Control<br>Number | Part                                              | HCPCS<br>Procedure<br>Code | Provider<br>Type | Type<br>of<br>Bill | Review<br>Decision | Error<br>Code | Fiscal<br>Year |
|--------|----------------------------|---------------------------------------------------|----------------------------|------------------|--------------------|--------------------|---------------|----------------|
| 146417 | 2268268                    | Part<br>A(Excluding<br>Inpatient<br>Hospital PPS) | NaN                        | SNF              | 212                | NaN                | Other         | 2022           |

In [24]: *# since this is a statistically meaningless portion of the data, I made the decision*

```
full_CERT_df = full_CERT_df.dropna(subset=['Review Decision'])
```

In [25]: *# changed Medicare Part column names for improved readability inside visualizations*

```
full_CERT_df['Part'] = full_CERT_df['Part'].replace({  
    'Part A(Excluding Inpatient Hospital PPS)': 'Part A Other',  
    'Part A(Inpatient Hospital PPS)': 'Part A Inpatient',  
    'Part B': 'Part B',  
    'DME MAC': 'DME MAC'  
})
```

While working with the Medicare DRG subset of claims data, I noticed that the Medical or Surgical column that was carried over from the Tuva Project data contained more values than anticipated. Most rows contained the expected "M" for medical and "P" for procedure/surgical; however, there were some rows where this column was either null or contained the value "Surgical."

In [26]: `DRG_subset_df['Medical or Surgical'].unique()`

Out[26]: `<StringArray>  
['M', 'P', 'Surgical', nan]  
Length: 4, dtype: str`

In [30]: `(DRG_subset_df['Medical or Surgical'] == 'Surgical').sum()`

Out[30]: `np.int64(1126)`

In [31]: `DRG_subset_df[DRG_subset_df['Medical or Surgical'].isnull()]`

Out[31]:

|       | Claim<br>Control<br>Number | Part                                    | HCP<br>Procedure<br>Code | Provider<br>Type     | Type<br>of<br>Bill | Review<br>Decision | Error<br>Code        | Fiscal<br>Year | MS-<br>DRG<br>Code |
|-------|----------------------------|-----------------------------------------|--------------------------|----------------------|--------------------|--------------------|----------------------|----------------|--------------------|
| 3130  | 2145556                    | Part<br>A(Inpatient<br>Hospital<br>PPS) | NaN                      | DRG<br>Short<br>Term | 111                | Agree              | -                    | 2021           | 132                |
| 8754  | 2161061                    | Part<br>A(Inpatient<br>Hospital<br>PPS) | NaN                      | DRG<br>Short<br>Term | 111                | Disagree           | Medical<br>Necessity | 2021           | 131                |
| 10126 | 2169461                    | Part<br>A(Inpatient<br>Hospital<br>PPS) | NaN                      | DRG<br>Short<br>Term | 111                | Agree              | -                    | 2021           | 129                |
| 11568 | 2234337                    | Part<br>A(Inpatient<br>Hospital<br>PPS) | NaN                      | DRG<br>Short<br>Term | 111                | Agree              | -                    | 2022           | 131                |



According to the CMS MS-DRG Definitions Manual, "M" designates a medical MS-DRG and "P" designates a surgical MS-DRG. The presence of "Surgical" as a third value was investigated by cross-referencing the raw Tuva Project reference file.

Filtering the Tuva reference data to rows containing the "Surgical" label revealed that all affected DRG codes carried a deprecated\_date of October 1, 2023, indicating they were retired in the FY2024 MS-DRG update. Examination of the descriptions showed that CMS consolidated several cardiac defibrillator and cardiovascular procedure DRGs, replacing the more granular "Surgical"-labeled codes with simplified "P"-labeled equivalents.

Based on this investigation, "Surgical" was replaced with "P" for consistency with current CMS MS-DRG classification standards. Additionally, 4 rows with null values in the Medical or Surgical column were dropped as they could not be meaningfully classified.

```
In [32]: DRG_subset_df['Medical or Surgical'] = DRG_subset_df['Medical or Surgical'].replace('Surgical': 'P')
})
```

```
In [33]: DRG_subset_df = DRG_subset_df.dropna(subset=['Medical or Surgical'])
```

In reviewing the MDC Code column for visualization, I noticed some values were null. Based on the ICD-10-CM/PCS MS-DRG v43.0 Definitions Manual, MS-DRG categories 1-19 represent Pre-MDC DRGs — extremely resource-intensive cases such as organ transplants, ECMO, and tracheostomies that are assigned directly to a DRG before MDC grouper logic is applied, and therefore do not belong to any MDC category.



```
In [37]: (DRG_subset_df['MDC Code'].isnull()).sum()
```

```
Out[37]: np.int64(1447)
```

The MDC Code column will be converted to string type to accommodate the text label, and rows where MS-DRG Code is 1-19 will be assigned the value "Pre-MDC" for accurate representation in visualizations.

```
In [ ]: # add Pre-MDC values for the appropriate MS-DRG codes
```

```
pre_mdc_drgs = [str(i) for i in range(1, 20)]  
DRG_subset_df.loc[DRG_subset_df['MS-DRG Code'].isin(pre_mdc_drgs), 'MDC Code'] = 'P
```

```
In [55]: # verify it worked
```

```
print("Nulls remaining:", DRG_subset_df['MDC Code'].isnull().sum())  
print("Pre-MDC count:", (DRG_subset_df['MDC Code'] == 'Pre-MDC').sum())
```

```
Nulls remaining: 561
```

```
Pre-MDC count: 886
```

```
In [56]: DRG_subset_df[DRG_subset_df['MDC Code'].isnull()]['MS-DRG Code'].unique()
```

```
Out[56]: <StringArray>  
['982', '988', '981', '987', '983', '989']  
Length: 6, dtype: str
```

Six additional MS-DRG codes (981, 982, 983, 987, 988, 989) also contained null MDC values. These represent O.R. Procedures Unrelated to the Principal Diagnosis — cases where a significant operating room procedure was performed but was not related to the principal diagnosis driving the admission. Per the CMS MS-DRG Definitions Manual Appendix F, these DRGs are also excluded from standard MDC grouper logic and do not receive an MDC assignment. These rows were assigned the value "Unrelated O.R. Procedures" for accurate representation in visualizations.

```
In [ ]: # add Unrelated O.R. Procedures for the identified MS-DRG codes
```

```
unrelated_or_drgs = ['981', '982', '983', '987', '988', '989']  
DRG_subset_df.loc[DRG_subset_df['MS-DRG Code'].isin(unrelated_or_drgs), 'MDC Code']
```

```
In [68]: # verify there are no more nulls
```

```
print("Nulls remaining:", DRG_subset_df['MDC Code'].isnull().sum())
```

```
Nulls remaining: 0
```

Now I want to add a column that describes the MDC codes, to provide immediate context to the viewer of the data. The source manual lists these descriptions including "Disease and Disorders of" in front of most of the values; I have chosen to remove this syntax from the values in the description column in order to improve readability in the final dashboard.

```
In [71]: # add description column

mdc_descriptions = {
    '01': 'Nervous System',
    '02': 'Eye',
    '03': 'Ear, Nose, Mouth and Throat',
    '04': 'Respiratory System',
    '05': 'Circulatory System',
    '06': 'Digestive System',
    '07': 'Hepatobiliary System & Pancreas',
    '08': 'Musculoskeletal System & Connective Tissue',
    '09': 'Skin, Subcutaneous Tissue & Breast',
    '10': 'Endocrine, Nutritional and Metabolic',
    '11': 'Kidney & Urinary Tract',
    '12': 'Male Reproductive System',
    '13': 'Female Reproductive System',
    '14': 'Pregnancy, Childbirth & the Puerperium',
    '15': 'Newborns and Other Neonates with Conditions Originating in Perinatal Per',
    '16': 'Blood, Blood Forming Organs and Immunologic Disorders',
    '17': 'Myeloproliferative Diseases & Disorders, Poorly Differentiated Neoplasms',
    '18': 'Infectious and Parasitic Diseases, Systemic or Unspecified Sites',
    '19': 'Mental',
    '20': 'Alcohol or Drug Use or Induced Organic Mental Disorders',
    '21': 'Injuries, Poisonings & Toxic Effects of Drugs',
    '22': 'Burns',
    '23': 'Factors Influencing Health Status and Other Contacts with Health Service',
    '24': 'Multiple Significant Trauma',
    '25': 'Human Immunodeficiency Virus Infections',
    'Pre-MDC': 'Pre-MDC',
    'Unrelated O.R. Procedures': 'Unrelated O.R. Procedures'
}

DRG_subset_df['MDC Description'] = DRG_subset_df['MDC Code'].map(mdc_descriptions)
```

```
In [65]: DRG_subset_df['MDC Description'].isnull().sum()
```

```
Out[65]: np.int64(0)
```

---

## Save the Updated Cleaned Datasets

---

```
In [72]: full_CERT_df.to_csv("cleaned_data_full_CERT.csv", index=False)
```

```
DRG_subset_df.to_csv("cleaned_data_DRG_sub.csv", index=False)
```

```
In [35]: print("Data cleaning complete. Cleaned CSV files created.")
```

Data cleaning complete. Cleaned CSV files created.